World Models in Action

4. Predicting in representation space

This chapter covers

- The JEPA objective and the transition from image to video prediction
- I-JEPA, V-JEPA, V-JEPA 2, and action-conditioned latent world models
- Collapse prevention, LeJEPA/SIGReg, and VISReg
- LeCun's proposed architecture for autonomous machine intelligence

A world model must retain information that helps an agent anticipate what can happen next, distinguish consequences that matter, and evaluate possible actions before committing to them. Chapter 2 framed this as a state-representation problem. Chapter 3 examined how an encoder can learn useful features without state labels. This chapter turns to a more direct training signal: predict the representation of something that is not currently available to the model.

Joint-Embedding Predictive Architectures (JEPAs) encode a context and predict a target in a shared representation space. The target may be a hidden region of an image, a masked portion of video, a future frame, or a text embedding associated with a visual context. The prediction is evaluated in feature space rather than by reconstructing every pixel. The design changes what the model is asked to explain: pixel reconstruction rewards texture, lighting and fine detail even when these are not relevant to the prediction of interest. A latent predictor can instead focus on aspects of the target that are predictable from the context under the selected representation.

The word *predictive* needs care. In an image JEPA, a target region can be spatially withheld without being in the future. In a video JEPA, a target can be temporally later, spatially masked, or both. In an action-conditioned model, the predicted representation depends on a proposed action as well as the current observation. A JEPA is therefore not necessarily a temporal model, and a temporal JEPA is not automatically an action model.

The chapter proceeds in this order: the generic objective, image and video examples, distributional regularization, action-conditioned prediction, and finally the broader autonomous-agent proposal described by LeCun (2022). This progression teaches the predictive component before placing it inside a complete architecture.

A world model is one component of an agent

An autonomous agent requires at least four capabilities that should not be conflated: a representation of the current situation; a model of how it may change; a way to assess outcomes against goals or costs; and a mechanism for selecting and executing actions. Memory and task

configuration affect all four. A representation-learning objective may improve the first two, but does not by itself provide a reliable planner, cost function, controller, or memory system.

LeCun's *A Path Towards Autonomous Machine Intelligence* proposes one way these pieces could fit together. It is a position paper, not a report of one fully implemented system. Its six top-level modules are Perception, World Model, Cost, Actor, Short-Term Memory, and Configurator. The Cost module contains an Intrinsic Cost component and a Trainable Critic; these are not additional top-level modules. We preview the module names here and reserve the detailed relationships for Section 4.3, after action-conditioned prediction.

A representation that ignores the position of a moving object may be adequate for scene classification but inadequate for predicting a collision. A representation that preserves object identity but not velocity may support recognition while failing to distinguish histories with different futures. The training target, mask and evaluation protocol determine which distinctions are encouraged.

Figure 4.1 - From an encoder to an agent.

> **Figure description.** Arrange four stages from left to right: observation encoder, latent predictor, evaluator of predicted outcomes, and actor that selects an action. Beneath them, show memory and task configuration as cross-cutting support modules rather than layers in the prediction network. Distinguish internal information flow from the action sent to the environment and the new observation returned by it. The argument is that JEPA supplies a predictive representation mechanism, not the entire agent.

Evidence boundaries

A mathematical result is valid under its stated assumptions. An empirical result belongs to the paper's data, architecture, training recipe and evaluation protocol. An architectural proposal may guide research, but is not established merely because its diagram is coherent.

I-JEPA demonstrates image representation learning with latent prediction. V-JEPA demonstrates feature prediction as a video pretraining objective. V-JEPA 2 separates action-free video pretraining from a later action-conditioned model trained with robot trajectories. LeJEPA develops a theory and objective for regularized embeddings. VISReg proposes a different regularizer separating scale and shape. These are related contributions, not interchangeable implementations of one fixed model.

Chapter 3 has already introduced contrastive learning, bootstrap methods, redundancy reduction, VICReg, masked reconstruction and collapse. We use those ideas as context rather than repeating

their full derivations. The focus here is the predictive target, the encoder-predictor system, and the distributional constraints placed on its representation.

4.1. Why predict in latent space?

A predictive model can estimate an observation, a distribution over observations, or a representation of an observation. These choices define different learning problems. A JEPA takes the third route: encode the context and target, then train a predictor to make the context representation agree with the target representation.

Let an observation be x, with representation $z = f_\theta(x)$. The objective is not to reconstruct x from z. Instead, use an available context x_c to predict a target feature z_t extracted from a withheld or later portion of the data. The loss measures the difference between the predicted and target embeddings.

This can be useful when the observation contains details that are expensive to model and irrelevant to the prediction of interest. A video frame contains texture, sensor noise, shadows and fine-grained appearance. Some details matter for physical interaction, but not all need to be predicted at every step. A model can potentially devote more capacity to object arrangement, motion and other structure by predicting in a feature space rather than reconstructing all pixels.

The qualification is important. The latent space does not become semantic merely because the predictor operates in it. The target encoder, mask distribution, temporal sampling, representation dimension, objective and regularizer determine what the model is encouraged to preserve. Latent prediction changes the learning target; it does not certify sufficiency for every downstream problem.

4.1.1. Prediction is not reconstruction

Suppose a context is compatible with several future frames. Pixel-level regression can average incompatible appearances. Latent prediction can focus on a target feature that captures predictable structure, leaving unpredictable detail outside the target. This may make learning more tractable, but it raises a design question: which aspects of the future count as predictable and important?

Under squared error, the optimal predictor of a target feature Z_t given context C is its conditional mean:

$$[g^*(C) = \mathbb{E}[Z_t \mid C].$$

For a candidate prediction (hC), add and subtract the conditional mean (mu$C$):

$$[\ \mathbb{E}[\ \| Z_t - h(C) \|^2 \mid C] = \mathbb{E}[\ \| Z_t - \mu(C) \|^2 \mid C] + \| \mu(C) - h(C) \|^2 .$$

The cross term vanishes because $\mathbb{E}[Z_t - \mu(C\mathrm{C}]=0)$. The first term does not depend on h, so the conditional mean minimizes expected squared error, assuming a finite conditional second moment.

This identity explains an important limitation. If the target distribution is multimodal, the conditional mean can lie between plausible futures. A deterministic predictor trained with a pointwise squared loss may represent an average rather than preserve the alternatives. A JEPA objective alone does not solve multimodal prediction or calibrated uncertainty.

4.1.2. The target determines the question

A target can be selected spatially, temporally, action-conditionally or across modalities.

- A spatial target asks what can be inferred about a hidden region from visible context.
- A temporal target asks what future content is predictable from earlier observations.
- An action-conditioned target asks what future representation is predicted after a particular action or action sequence.
- A cross-modal target asks which representation in one modality is compatible with a context from another modality.

These targets are not equivalent. Predicting a masked patch in the same frame does not test whether the model predicts motion. Predicting future video without actions does not identify the consequences of a candidate control input. Predicting text embeddings from an image does not by itself provide a physical dynamics model.

The mask is part of the task definition. A target too close to visible context may admit a shortcut based on local texture. A target too distant or too large may be difficult to infer. A context that hides too much may remove information necessary for prediction. Success depends on the relation between what is visible and what is withheld.

4.1.3. The representation contract remains in force

Chapter 2 defined a useful state in terms of the information needed for downstream prediction and control. A JEPA offers a way to learn such a representation, but its pretraining loss is still a proxy objective. We must ask whether the representation retains distinctions that affect relevant futures and whether downstream probes or tasks can access them.

A frozen linear probe measures information accessible to a linear readout under a chosen protocol. A nonlinear probe tests a more flexible readout. A dense prediction task tests spatial structure. A planning experiment evaluates a model together with a planner, cost, action parameterization and environment. None establishes universal predictive sufficiency.

Figure 4.2 - Three different prediction targets.

> **Figure description.** Show an image split into visible context and hidden spatial block; a video split into observed and future segments; and an action-conditioned transition in which one current state branches under two actions. In each panel, show a context encoder, target encoder and predictor. Label the first target spatial, the second temporal, and the third action-conditioned. The drawing should show that the JEPA mechanism is similar while the prediction question changes.

Representation-space prediction is a learning strategy, not a guarantee of semantic structure. Its value depends on choosing a target that exposes useful regularities without requiring the model to predict every observable detail.

4.2. The joint-embedding predictive objective

A canonical JEPA contains a context encoder, a target encoder and a predictor. The context encoder processes the part of the observation that the model is allowed to see. The target encoder produces features for the withheld or later target. The predictor maps context features, together with information about the requested target, to an estimate of those target features.

Let x_c denote the context and x_t the target. Their embeddings are

$$[z_c = f_\theta(x_c), \qquad z_t = f_{\bar{\theta}}(x_t).$$

A predictor with parameters ϕ produces

$$[\hat{z}_t = g_\phi(z_c, q),$$

where q identifies the target location, time or query. A useful abstract objective is

$$[\mathcal{L}_{pred} = \mathbb{E}_{(x_c,\, x_t,\, q)}\Big[d\big(g_\phi(f_\theta(x_c), q),\, (f_{\bar{\theta}}(x_t))\big)\Big].$$

Here d is a feature-space discrepancy and [stem b350fb515bfaa32c3060a966f22aae77] denotes stop-gradient on the target branch. This is a generic schematic, not a universal definition of every JEPA implementation.

In common teacher-student designs, the target parameters are updated as an exponential moving average:

$$[\bar{\theta} \leftarrow \tau\bar{\theta} + (1-\tau)\theta,$$

where τ is close to one. EMA provides a slowly changing target. It is a common stabilizing choice, not a requirement of the JEPA family. LeJEPA, for example, uses a different multi-view design

without an EMA teacher.

4.2.1. The query is part of the prediction

The predictor needs to know which target it should estimate. For an image, this may be a spatial position associated with a masked block. For video, it may include spatial and temporal coordinates. For action-conditioned prediction, actions and relevant robot state are additional inputs.

A pooled context vector alone may not specify whether the model should estimate a target on the left or right of an image, or at which time step. A query supplies this information. Transformer implementations often combine learned mask or query tokens with positional encodings.

The predictor is not simply a decoder with a pixel loss. Its output is an embedding under the target encoder's representation convention. It may itself be a substantial transformer, but it is trained to predict features rather than observations.

4.2.2. What makes the embedding spaces joint?

The context and target embeddings must be comparable. They may be produced by separate networks with related parameterization, or by a shared encoder applied to different inputs. In teacher-student designs, the target network changes more slowly than the context network, providing a relatively stable reference.

The branches receive different information. The context encoder cannot see target content, while the target encoder may process the full observation or the unmasked target crop. The asymmetry makes prediction non-trivial: the model must infer target features from context rather than copy them from the input.

The target-update rule matters. If targets move too quickly, the predictor chases a changing representation. If targets move too slowly, they can become stale. EMA schedules, normalization, predictor depth and optimization are implementation decisions to be checked against the method being discussed.

4.2.3. The loss does not specify the whole method

A complete recipe must specify how contexts and targets are sampled; whether targets are blocks, frames, tokens or modalities; how the target encoder is updated; how many targets are predicted from one context; how target positions are represented; how the loss is normalized and aggregated; how collapse is discouraged; and which representation is evaluated downstream.

A simple loss can hide substantial inductive bias in the sampling procedure. I-JEPA demonstrates this with its spatial block-mask design. V-JEPA extends the target to video. V-JEPA 2 adds action-

conditioned prediction only after action-free pretraining. LeJEPA changes the regularization strategy. These distinctions are central to the methods.

4.2.4. Collapse and regularization

A constant representation can satisfy some prediction objectives without retaining useful variation. Architectural asymmetry, target networks, stop-gradient, normalization and explicit regularizers are different responses to this issue. Chapter 3 already developed the main anti-collapse families, so this chapter only needs a brief handoff.

For a batch of embeddings $Z \in \mathbb{R}^{B \times D}$, a variance constraint discourages individual dimensions from becoming constant. A covariance penalty discourages redundant linear correlations. These constraints are useful but do not determine the full joint distribution: zero covariance does not imply independence or Gaussianity.

LeJEPA uses projected distributional tests to compare embeddings with an isotropic Gaussian target. VISReg instead combines explicit scale control, a sliced-Wasserstein shape term and centering. Their statistical targets and gradient paths differ, so they need separate treatments.

Figure 4.3 - A canonical JEPA training step.

> **Figure description.** Divide an image or video into visible context and hidden target. Route visible content through the context encoder and target content through a slowly updated target encoder. Feed context features and a target query to the predictor, then compare its output with a stopped-gradient target embedding through a feature-space loss. Draw a dashed EMA update arrow from context to target encoder. Do not show a pixel decoder or reconstruction loss.

A low prediction loss establishes agreement with the chosen target under the chosen sampling distribution. It does not establish that the representation contains every detail needed by another task, that predicted features correspond to physically valid states, or that uncertainty is represented correctly. Action-conditioned prediction additionally needs action information and action-state transitions in its training data.

4.3. I-JEPA: predicting hidden regions of an image

I-JEPA is the image-based JEPA introduced by Assran and colleagues (2023). It predicts representations of several target blocks from a single context block in the same image. The target is not a patch of pixels reconstructed by a decoder; it is a feature representation produced by a target encoder.

The image is divided into patch tokens by a Vision Transformer. A context mask selects a spatially distributed subset of patches. The context encoder processes those visible tokens. Multiple target masks select larger blocks elsewhere in the image. The target encoder processes the corresponding target regions and supplies the reference features. The predictor receives the context representation and a query for each target position, then predicts the target features.

The target encoder is initialized from the context encoder and updated by an exponential moving average. Its outputs are treated as targets rather than as a branch directly optimized by the prediction loss. The predictor estimates multiple target blocks from the same context. Since the context is spatially distributed, it can contain evidence about objects and scene structure beyond a single contiguous crop.

4.3.1. Why the mask matters

If target blocks are too small, the model may exploit local continuity or texture. If the visible context is too narrow or poorly positioned, important evidence may be hidden. If the targets are too large, they may contain too much unpredictable content. The I-JEPA paper emphasizes two connected choices: target blocks should be sufficiently large to require semantic context, and the context should be spatially distributed and informative.

The model does not receive a target image patch through a pixel decoder. It receives target embeddings from the target encoder. This makes that encoder's representation central to the method. The target may ignore variation that does not affect its features, but it can also omit information if its representation is insensitive to it.

4.3.2. What the experiments establish

The CVPR 2023 paper reports a ViT-Huge/14 trained on ImageNet using 16 A100 GPUs in under 72 hours. The reported evaluation includes classification, object counting and depth prediction. This supports the claim that a non-generative feature-prediction objective can learn useful image representations under the paper's setup.

It does not establish physical dynamics. A single image does not identify velocity without temporal information, and the method does not learn how an action changes a scene. I-JEPA is an image representation learner and an example of spatial latent prediction, not an action-conditioned world model.

Figure 4.4 - I-JEPA's context and multiple target blocks.

> **Figure description.** Show an image partitioned into patch tokens. Highlight one spatially distributed context mask and two large target blocks elsewhere. On the right, show the context encoder producing context tokens, an EMA target

> encoder producing target features, and a predictor receiving two sets of position-aware query tokens. Draw a separate feature-space loss for each target block. Do not draw a pixel reconstruction decoder. Emphasize the relation between visible context and multiple withheld targets.

I-JEPA is the clearest starting point for JEPA's spatial prediction mechanism. V-JEPA makes the target temporal by moving from images to video.

4.4. V-JEPA: predicting features from video

V-JEPA extends feature prediction from images to video. In the original work, Bardes and colleagues (2024) train visual representations with a feature-prediction objective on video, without a pretrained image encoder, text, negative examples, pixel reconstruction or other external supervision. The central change from I-JEPA is the temporal structure of the input and target.

A clip can be represented as a grid of spatiotemporal tokens. The context contains visible portions of the clip, while the target contains masked spatiotemporal regions. A context encoder processes visible tokens. A target encoder produces features for hidden regions. A predictor estimates those features using context information and the target's spatial and temporal location.

An individual image contains appearance and spatial arrangement at one instant. A sequence contains change. An object at one location may be moving, stationary or about to be occluded. These possibilities can look similar in one frame but imply different future observations. Video prediction can therefore produce features sensitive to appearance and motion.

The target is not necessarily a later frame. V-JEPA's masked spatiotemporal prediction objective withholds regions across space and time. The model predicts features for those regions rather than reconstructing every pixel. The target representation defines the level of abstraction at which prediction is rewarded.

4.4.1. The original empirical evidence

The original V-JEPA paper reports training on approximately two million videos collected from public datasets. Its largest model, a ViT-H/16 trained only on videos, obtains 81.9% on Kinetics-400, 72.2% on Something-Something-v2 and 77.9% on ImageNet-1K with a frozen backbone under the paper's protocols. These results show transfer to both motion-centric and appearance-centric evaluations.

A frozen-backbone score measures feature quality under a particular readout, not the ability to plan an action sequence. V-JEPA studies representation learning. It does not automatically infer the causal effect of a robot command: passive video records observed motion but does not necessarily

expose the controls that produced it.

Schematically, passive video prediction uses context such as $z_{\le t}$, whereas action-conditioned prediction also receives a_t:

$$p(z_{t+1} \mid z_{\le t}) \quad \text{versus} \quad p(z_{t+1} \mid z_{\le t}, a_t).$$

These expressions identify the information difference; they are not literal definitions of the V-JEPA or V-JEPA 2 objectives.

Figure 4.5 - From spatial masking to spatiotemporal masking.

> **Figure description.** Draw a short clip as a sequence of frame grids. Mark visible context tokens across several frames and masked target tubes spanning space and time. Route visible tokens through the context encoder and masked target tokens through the target encoder, then show the predictor estimating target features at spatiotemporal positions. Add a small inset showing that the target is a feature tensor, not reconstructed RGB pixels. Emphasize that the JEPA pattern now captures temporal regularities.

V-JEPA shows that predicting video features can learn motion-sensitive visual representations. Predicting consequences of an action requires action information and a new training stage.

4.5. V-JEPA 2 and V-JEPA 2.1

V-JEPA 2 makes the transition from video representation learning toward world modeling more explicit. It separates large-scale action-free pretraining from an action-conditioned post-training stage. The pretrained encoder learns visual structure from images and videos, while the action-conditioned predictor is trained with robot trajectories to model how future visual states depend on actions.

The original V-JEPA 2 paper reports pretraining on a video and image dataset comprising more than one million hours of internet video. It describes 22 million samples combining video and images. The pretrained model is action-free: it learns from visual observations without requiring action labels for every sequence.

4.5.1. The action-free stage

The encoder and predictor are pretrained with masked latent feature prediction. As in earlier JEPAs, the model is not trained to reconstruct RGB pixels. The paper reports 77.3 top-1 accuracy on Something-Something-v2 for motion understanding and 39.7 recall-at-five on EPIC-KITCHENS-100 for human action anticipation. It also reports video question-answering

evaluations after aligning the visual encoder with a language model. Those VQA scores should not be conflated with the planning evaluation.

4.5.2. V-JEPA 2-AC: adding actions

After pretraining, the authors freeze the visual encoder and learn a new action-conditioned predictor. It estimates future frame representations from past visual features, robot actions and end-effector states. The paper uses an autoregressive feature-prediction objective with a teacher-forced prediction loss and an additional rollout loss.

A schematic model is

$$\hat{z}_{t+1} = g_\phi(z_{\le t}, a_{\le t}, s_{\le t}),$$

where z denotes visual features, a actions and (s) end-effector state. The actual model uses sequences and specific action/state encodings. The key point is that the predictor receives variables describing the intervention, not only the observed video.

At inference, the system receives a goal image and plans an action sequence expected to reach a matching visual state. The authors report deployment on Franka arms in two laboratories for reaching, grasping and pick-and-place tasks. They describe this as zero-shot with respect to the test environments: no data were collected from those deployment environments, and no task-specific training or reward was required for the evaluated tasks. The system did use robot data: the action-conditioned stage used less than 62 hours of unlabeled robot video from the DROID dataset.

`Zero-shot'' therefore describes the relation between training and evaluated environments or tasks. It should not be read as` no interaction data," `no training,'' or` no planning."

4.5.3. Planning in representation space

A planner can search over candidate action sequences by evaluating the future representations they imply. A goal image is encoded into a target representation. Candidate actions are rolled through the action-conditioned model. The predicted representation is compared with the goal representation and the action sequence is updated.

A schematic objective is

$$\hat{\mathbf{a}}_{1:H} = \arg\min_{\mathbf{a}_{1:H}} d(\hat{z}_{t+H}(\mathbf{a}_{1:H}), z_{goal}) + \lambda R(\mathbf{a}_{1:H}),$$

where (d) measures agreement with a goal representation and ® can express an action or feasibility penalty. This explains the logic of latent-space planning; it is not a verbatim statement of every term in the V-JEPA 2-AC implementation.

A predicted goal match can be plausible in feature space while the physical state is unsafe or infeasible. Planning quality depends on the representation, action model, horizon, cost, action constraints and feedback loop. Results from two laboratories are evidence under the reported setup, not proof of general-purpose physical reasoning.

4.5.4. V-JEPA 2.1 is a different extension

V-JEPA 2.1 focuses on dense visual features rather than introducing the action-conditioning stage. Its paper describes four components: a dense predictive loss supervising visible and masked tokens, deep self-supervision across intermediate encoder layers, multimodal tokenizers for image and video inputs, and model/data scaling. The stated motivation is to improve spatial and temporal structure in feature maps while retaining global scene understanding.

V-JEPA 2.1 should not be described as synonymous with V-JEPA 2-AC. The former is a dense representation-learning recipe. The latter is the action-conditioned world model introduced in the V-JEPA 2 paper. The official repository maintains code for V-JEPA 2, 2-AC and 2.1, but their training objectives and system roles differ.

Figure 4.6 - The two-stage V-JEPA 2 system.

> **Figure description.** Show web videos and images feeding an action-free visual encoder and predictor. Draw a frozen-backbone arrow into a second training stage receiving robot video, action vectors and end-effector states. Show the second predictor producing future latent states. At deployment, route a goal image and current observation to a planner that proposes actions, rolls them through the predictor and sends the first action to a robot. Label the training-data and deployment-environment boundaries.

Figure 4.7 - V-JEPA 2.1's dense and deep supervision.

> **Figure description.** Draw a video transformer with several intermediate encoder layers. From selected layers, show feature maps containing visible and masked tokens, both contributing to a dense predictive objective. Include a separate tokenizer path for images and videos and a scaling arrow for data and model capacity. Emphasize that dense feature supervision is distinct from action-conditioned future prediction.

4.6. From collapse prevention to distributional regularization

A predictive loss does not automatically determine a useful embedding geometry. If every input maps to the same vector, a representation can contain no information about the input. Some architectures avoid this solution through asymmetry and optimization dynamics; other methods impose explicit statistical constraints on the batch of embeddings.

Chapter 3 has already explained the mechanisms used by contrastive, bootstrap, self-distillation and redundancy-reduction methods. This section only establishes the transition needed for LeJEPA and VISReg.

4.6.1. Three different questions

It is useful to separate three properties of an embedding distribution:

1. **Non-collapse:** do different inputs produce a non-constant representation?
2. **Redundancy:** do feature dimensions carry strongly overlapping linear information?
3. **Distributional shape:** what is the joint distribution of the embedding vectors, beyond their coordinate-wise variance and pairwise covariance?

VICReg addresses the first two explicitly. Its variance term maintains a minimum standard deviation for each coordinate, while its covariance term penalizes off-diagonal covariance. The method also contains an invariance term that brings representations of different views of the same sample together. These components are not interchangeable: variance prevents individual dimensions from becoming constant; covariance discourages linear redundancy; invariance defines which views should agree.

Barlow Twins instead uses the cross-correlation matrix between two batches of embeddings and pushes it toward the identity. Its diagonal terms align matching coordinates across views, while off-diagonal terms penalize cross-coordinate redundancy. This is not the same mechanism as VICReg's within-view variance floor or EMA teacher updates. Both methods belong to the broader family of explicit statistical regularization, but their statistics and optimization behaviour differ.

4.6.2. Why second-order statistics leave a gap

Mean and covariance do not specify an arbitrary multivariate distribution. Two distributions can share the same mean and identity covariance while differing in skewness, tail behaviour, multimodality or support. Uncorrelated coordinates need not be independent, and uncorrelated features are not necessarily Gaussian.

This observation motivates distributional regularizers. Instead of constraining only a few moments, they compare the embedding distribution with a target distribution using a statistic that can detect a

broader range of differences.

That additional target is also a stronger modeling assumption. If the regularizer asks embeddings to follow an isotropic Gaussian, it is not merely asking that the model avoid collapse. It is prescribing a shape. Whether that shape is useful depends on the downstream criterion and the training setup. LeJEPA motivates its Gaussian target through a particular downstream prediction-risk analysis. VISReg adopts a Gaussian shape target as part of a scale-and-shape regularizer. Neither should be described as proving that Gaussian embeddings are universally ideal for every representation task.

4.6.3. The handoff to SIGReg and VISReg

LeJEPA's SIGReg sketches one-dimensional distributional tests along sampled directions and compares each projected feature distribution with a standard normal. Its statistic is based on empirical characteristic functions. VISReg uses a different sketch: it sorts projected samples and compares them with Gaussian quantiles using a one-dimensional Wasserstein objective. It also separates the scale correction from the shape calculation.

Both methods rely on projections, but their losses, gradient paths and finite-sample behaviour are not the same. The next two sections derive them independently.

Figure 4.8 - Same covariance, different distributional shape.

> **Figure description.** Show three two-dimensional point clouds with zero mean and identity covariance: an approximately Gaussian cloud, a symmetric four-lobed or cross-shaped distribution, and a multi-cluster distribution arranged to retain the same covariance. Mark coordinate axes and indicate that variance and covariance alone cannot distinguish all three distributions. Label the figure as a conceptual illustration, not an empirical result. The point is that decorrelation constrains second-order statistics but does not determine the full joint density.

4.7. LeJEPA and Sketched Isotropic Gaussian Regularization

LeJEPA, introduced by Balestriero and LeCun (2025), combines a predictive multi-view objective with a distributional regularizer called Sketched Isotropic Gaussian Regularization (SIGReg). Its central proposal is that, under the downstream prediction-risk framework studied in the paper, an isotropic Gaussian embedding distribution is optimal. SIGReg then provides a practical way to encourage that distribution without evaluating a high-dimensional density directly.

The theorem and the implementation answer different questions. The theory studies what embedding distribution minimizes a specified downstream risk. SIGReg is a stochastic numerical

objective that encourages the learned embeddings to approach that target. The latter does not turn a finite minibatch into a certificate that the population embedding distribution is exactly Gaussian.

4.7.1. The Gaussian target and the scope of the theory

Let an encoder produce an embedding $Z = f_\theta(X) \in \mathbb{R}^d$. The paper studies downstream prediction from frozen representations. In its linear setting, a probe fits a regularized least-squares estimator. The analysis compares embedding geometries under matched representational capacity, including a comparison between isotropic and anisotropic covariance structures. Anisotropy can increase the worst-case bias of a regularized linear estimator and the variance of an unregularized estimator.

The paper also analyzes nonlinear probes, including radius-based nearest-neighbour regression and kernel regression. Under the regularity conditions and fixed-variance constraints stated in the paper, the analysis identifies the isotropic Gaussian as the unique minimizing distribution for the risk terms considered. The Gaussian's score function and the associated Fisher-information functional are central to the nonlinear analysis.

This is a meaningful theoretical result, but its scope must remain explicit. It concerns the downstream estimator families, risk criterion, regularity assumptions and fixed-scale constraints used in the paper. It is not a theorem that Gaussian latents are optimal for every conceivable task, every control problem, every conditional transition model or every data distribution. A task may need discrete modes, constraints, multimodality or structured variables that a simple isotropic Gaussian prior does not express on its own.

The useful design principle is therefore conditional: if the goal is to support the class of downstream estimators and risk assumptions analyzed in LeJEPA, isotropic Gaussian embeddings provide the paper's optimal target. The chapter should not turn that statement into an unrestricted claim about semantic quality or physical sufficiency.

4.7.2. From a multivariate test to one-dimensional projections

Suppose the empirical embedding distribution is P_θ and the desired target is $Q = \mathcal{N}(0, I_d)$. Directly testing whether a high-dimensional sample comes from Q can be computationally expensive. SIGReg instead projects the embeddings onto unit directions.

For a unit vector $a \in \mathbb{S}^{d-1}$, the scalar projection is

$$[U = a^\top Z.$$

If $Z \sim \mathcal{N}(0, I_d)$, then $U \sim \mathcal{N}(0, 1)$ for every unit direction a. A directional test can therefore compare the projected sample with a univariate standard-normal target.

The Cramér-Wold theorem supplies the identification principle: a multivariate probability

distribution is uniquely determined by the family of all its one-dimensional projected distributions. If every one-dimensional projection of P matches the corresponding projection of Q, the two distributions are equal. That is an all-directions statement.

SIGReg uses a finite collection of directions at each training step. It resamples directions during training, so the optimizer sees a changing set of one-dimensional views over time. This stochastic sketch is computationally practical, but a finite minibatch and a finite number of directions do not prove exact equality of the population distributions. The paper's analysis also studies how embedding smoothness and repeated random directions affect how strongly a finite sketch constrains the full distribution.

4.7.3. Characteristic functions

For a scalar random variable U, its characteristic function is

$$[\varphi_U(t) = \mathbb{E}[e^{itU}].$$

For a standard normal random variable,

$$[\varphi_{\mathcal{N}(0,\,1)}(t) = e^{-t^2/2}.$$

Given projected samples $u_1, \ldots, u_B$, the empirical characteristic function is

$$[\widehat{\varphi}_B(t) = \frac{1}{B}\sum_{b=1}^{B} e^{itu_b}.$$

The empirical characteristic function is a finite average of complex exponentials. It is differentiable with respect to the samples and can be aggregated across distributed workers using sum-reduction operations. Unlike high-order moments, the complex exponentials remain bounded as the magnitude of the input grows.

4.7.4. The Epps-Pulley statistic

LeJEPA uses an Epps-Pulley-style statistic to compare an empirical characteristic function with the target characteristic function:

$$[T_{EP}(u_{1:B}) = B\int_{-\infty}^{\infty} \left|\widehat{\varphi}_B(t) - \varphi_0(t)\right|^2 w(t)\,dt,$$

where $\varphi_0(t{=}e^{\{-t}2/2\})$ and $w(t)$ is a weighting function, typically Gaussian. The weighting controls how discrepancies across frequencies contribute to the statistic.

This formula is the population-integral description. A practical implementation approximates the integral by numerical quadrature over a finite set of frequencies. The LeJEPA paper reports a

trapezoidal approximation using a small number of integration knots, and its experiments ablate the range and number of points. It also discusses the finite-minibatch bias in the empirical statistic, which vanishes at an $\mathcal{O}(1/B)$ rate in the paper's analysis.

SIGReg samples directions $a_1, \ldots, a_M$, computes the projected samples $a_m^\top z_b$, evaluates a univariate statistic for each direction, and aggregates the directional losses. For the training objective, the paper averages directional statistics rather than taking their maximum. The maximum appears in the testing-theoretic construction; averaging is chosen in the optimization objective to avoid sparse gradients concentrated on only the worst direction.

A schematic training loss is

$$\mathcal{L}_{SIGReg}(Z) = \frac{1}{M}\sum_{m=1}^{M} T_{EP}\left(a_m^\top z_1, \ldots, a_m^\top z_B\right).$$

The precise quadrature range, frequency weights, direction sampling, normalization and distributed aggregation are implementation details and should be read from the cited paper version and official code, not inferred from this schematic.

4.7.5. The LeJEPA objective

LeJEPA uses multiple global and local views of an observation. Let $z_{n,v}$ denote the embedding for sample n and view v. Let the first V_g views be global, and let $V = V_g + V_l$ include all global and local views. The paper's prediction term encourages each view to agree with the mean embedding of the global views:

$$\mu_n = \frac{1}{V_g}\sum_{v=1}^{V_g} z_{n,v}, \qquad \mathcal{L}_{pred} = \frac{1}{V}\sum_{v=1}^{V} \| \mu_n - z_{n,v} \|_2^2 ,$$

averaged across samples. SIGReg is applied to the embedding sets for the views. The total objective is

$$\mathcal{L}_{LeJEPA} = (1-\lambda)\mathcal{L}_{pred} + \lambda\mathcal{L}_{SIGReg}.$$

This formulation differs from the common EMA teacher-predictor JEPA. In the specified LeJEPA recipe, the model does not require an EMA teacher, stop-gradient target branch or a predictor network. It uses a prediction term over multiple views and an explicit distributional regularizer. It should not be generalized into a claim that no JEPA requires target stabilization or that every target-prediction architecture can remove its predictor without changing the task.

4.7.6. Why characteristic functions?

The paper compares moment-based, cumulative-distribution and characteristic-function tests.

Moment tests can be sensitive to outliers and require higher moments to distinguish more distributional structure. CDF-based tests often involve sorting or order statistics, creating synchronization and differentiability difficulties in distributed gradient training. The characteristic function is differentiable and naturally amenable to averaging across data-parallel workers.

The numerical quadrature is an approximation. The Epps-Pulley statistic is the statistical object; the particular grid, frequency range and finite batch define its practical estimate. LeJEPA reports linear memory and computational complexity in minibatch size for its implementation, while the full cost also depends on embedding dimension, number of projected directions and quadrature points. Complexity statements must state which quantities are held fixed.

4.7.7. What SIGReg does and does not establish

SIGReg provides a differentiable distributional signal that goes beyond mean and covariance matching. It samples one-dimensional projections and compares their empirical characteristic functions with a normal target. The repeated sampling of directions is central to its sketching interpretation.

It does not establish exact Gaussianity from one training step. It does not guarantee that the representation is a sufficient statistic for every environment. It does not ensure calibrated uncertainty or action-conditioned dynamics. It does not prove that distances in latent space correspond to physical distances. And the theorem's downstream optimum should not be read outside its assumptions.

The LeJEPA paper reports stability across different architectures, datasets and hyperparameters, including scaling to very large models. It also reports that training loss correlates with linear-probe performance in the tested ImageNet setup. These are empirical findings of that paper, not a universal guarantee that training loss always replaces downstream evaluation.

Figure 4.9 - SIGReg from a latent batch to projected normality tests.

> **Figure description.** Draw a batch of high-dimensional embeddings as points in a cloud. Sample several unit directions from the sphere and project the same points onto each direction. For one selected slice, show scalar samples, their empirical characteristic function, the standard-normal characteristic function, and the weighted squared discrepancy across frequencies. A side panel should distinguish the ideal Cramér-Wold statement over all directions from the finite, resampled set used by SIGReg in training. Label the finite sketch as an approximation, not an exact population certificate.

Figure 4.10 - Mean and covariance do not determine the distribution.

> **Figure description.** Show two two-dimensional distributions with the same mean and identity covariance but visibly different shapes. Under each, plot several one-dimensional projections. Mark a direction where the projected distributions differ. The intended point is that second-order matching can miss higher-order shape differences, while directional distribution tests can detect them when an informative projection is sampled.

The next section examines a related but distinct approach. VISReg retains a scale constraint and replaces the characteristic-function discrepancy with a sliced-Wasserstein shape objective.

4.8. VISReg: separating scale and shape

VISReg (Wu, Balestriero and Levine, 2026) proposes Variance-Invariance-Sketching Regularization for JEPA training. It is motivated by a distinction between the magnitude of a representation and the shape of its distribution. The method retains an explicit scale penalty and uses a sliced-Wasserstein objective to compare normalized projected embeddings with a Gaussian target.

The authors' motivation is that VICReg's covariance term controls second-order relationships but not the full distributional shape, while SIGReg's characteristic-function sketch can provide weak gradients near collapse in the tested setup. VISReg responds by separating scale and shape rather than asking one distributional discrepancy to perform both roles. These are the paper's motivation and empirical claims, not a proof that VISReg dominates every other regularizer on every model or dataset.

4.8.1. Center, scale and shape

Let $Z \in \mathbb{R}^{B \times D}$ be a batch of embeddings, with B samples and D feature dimensions. Define the batch mean

$$\mu = \frac{1}{B}\sum_{b=1}^{B} z_b, \qquad Z_c = Z - \mu.$$

Let $\sigma_j(Z_c)$ be the standard deviation of coordinate j. VISReg defines a scale term

$$\mathcal{L}_{scale} = \frac{1}{D}\sum_{j=1}^{D} \left(1 - \sigma_j(Z_c)\right)^2.$$

This term encourages unit standard deviation per coordinate. It supplies a direct penalty when the

embedding dimensions shrink toward zero.

The shape term first normalizes the centered embeddings by their coordinate-wise standard deviations. The standard deviation is detached from the gradient path:

$$\tilde{Z} = \frac{Z_c}{(\sigma(Z_c)) + \epsilon}.$$

Here [stem b350fb515bfaa32c3060a966f22aae77] is stop-gradient and the division is coordinate-wise. The small ϵ protects the normalization from numerical division by zero. The normalization is intended to remove scale from the shape comparison, so that the shape loss is not asked to restore the overall feature magnitude.

4.8.2. Sliced-Wasserstein shape objective

For a unit vector $w_k \in \mathbb{S}^{D-1}$, project the normalized embeddings:

$$u_k = \tilde{Z} w_k.$$

The projected samples form a one-dimensional empirical distribution. If the target is a standard isotropic Gaussian in the embedding space, each unit-direction projection has a standard-normal target distribution.

For equally weighted empirical samples with equal cardinality, the one-dimensional quadratic Wasserstein distance can be computed by sorting the samples and matching corresponding order statistics. Let q_B denote the vector of target standard-normal quantiles under the paper's finite-sample convention. The shape loss is

$$\mathcal{L}_{shape} = \frac{1}{K} \sum_{k=1}^{K} \| (\tilde{Z} w_k) - q_B \|_2^2.$$

The K directions are sampled and normalized. Sorting gives the one-dimensional optimal matching for equal sample weights and equal sample counts. Different weighting or quantile conventions would change the exact formula, so the figure and prose should not hide this assumption.

The Cramér-Wold theorem again provides the population-level intuition: all one-dimensional projections determine the multivariate distribution. VISReg uses a finite set of slices, so the training loss is a stochastic sketch. The number of directions K affects the quality of the approximation and the computation. It is not correct to say that finite K guarantees exact distributional equality.

4.8.3. Centering and total objective

VISReg adds a centering term

$$\mathcal{L}_{center} = \| \mu \|_2^2 .$$

The regularizer combines the three parts, with weights made explicit:

$$\mathcal{L}_{reg} = \lambda_s \mathcal{L}_{scale} + \lambda_h \mathcal{L}_{shape} + \lambda_c \mathcal{L}_{center}.$$

The full objective combines this regularization with a predictive invariance term:

$$\mathcal{L}_{VISReg} = (1 - \lambda)\mathcal{L}_{pred} + \lambda \mathcal{L}_{reg}.$$

The exact normalization, loss weights, view-averaging convention and standard-deviation implementation should follow the paper's version rather than a notationally simplified equation. In particular, the symbols K and D have different roles: D is the embedding dimension, while K is the number of random slices.

4.8.4. Why separate scale from shape?

The scale loss acts directly on the standard deviation. For one coordinate, its derivative is

$$\frac{\partial}{\partial \sigma_j} \frac{1}{D}(1 - \sigma_j)^2 = -\frac{2}{D}(1 - \sigma_j).$$

As $\sigma_j \to 0$, this derivative approaches $-2/D$, rather than zero. This demonstrates a non-vanishing derivative with respect to the scalar standard-deviation statistic at collapse.

It is important not to overstate what this derivative proves. It is not, by itself, a lower bound on the gradient with respect to every encoder parameter. The chain from parameters to batch statistics can be affected by the architecture, normalization, saturation, finite precision and data batch. The paper's gradient comparisons are experiments under a stated simulation and training setup; the displayed scalar derivative is an analytic explanation of the scale term, not a universal theorem of recovery for arbitrary networks.

The detached standard deviation in the shape normalization prevents the shape loss from sending gradients through the scale denominator. This separates the correction of feature magnitude from the distributional shape comparison. The stop-gradient here has a different role from the EMA target asymmetry in I-JEPA or V-JEPA. Do not conflate all stop-gradient operations into one mechanism.

4.8.5. Computational considerations

The main work includes projecting a batch into K directions and sorting B values along each direction. A schematic cost is

$$\mathcal{O}(BDK) + \mathcal{O}(KB\log B).$$

The first term is the projection work; the second is the per-direction sorting cost. This expression makes the dependence on the number of slices explicit. If K grows with D, the projection term changes accordingly. The paper discusses distributing slices across GPUs, which can increase the number of directions without requiring all projections to be handled by a single device.

The project page describes the method as linear in its scaling factors and highlights distributed slices. That statement should be read with the chosen scaling convention stated. It does not make the computation independent of batch size, embedding dimension or slice count.

4.8.6. What the experiments show

The authors report experiments on low-quality, long-tailed and low-rank data regimes, as well as out-of-distribution evaluation. They report strong OOD linear-probe performance at ImageNet-1K scale and comparable average OOD performance to DINOv2 in a comparison where VISReg is pretrained on ImageNet-22K and DINOv2 on the larger LVD-142M dataset. These comparisons depend on the exact model, data, metrics and benchmarks used in the paper.

The collapse-gradient comparison should be described as the authors' controlled experiment. It is not a proof that all SIGReg implementations will fail or that VISReg cannot collapse under any possible training condition. The strongest conclusion is narrower: within the studied setups, the explicit scale term provides a different gradient path near collapse, and the decoupled design showed useful empirical behaviour across the evaluated data regimes.

4.8.7. Comparing the regularizers

Method	Main distributional constraint	What it directly measures	Important boundary
VICReg	Per-coordinate variance and covariance	Scale and pairwise linear redundancy	Does not determine the full joint distribution
SIGReg	Projected characteristic-function discrepancy	Difference between projected empirical and target distributions	Finite projections and batches are stochastic estimates
VISReg	Separate variance scale term and sliced-Wasserstein shape term	Per-coordinate spread plus sorted projected-sample discrepancy	Finite slices, quantile convention and gradient path matter

Figure 4.11 - Scale and shape can fail separately.

> **Figure description.** Show three embedding clouds: one compressed toward the origin, one with adequate radius but strong anisotropy, and one with adequate coordinate-wise spread but non-Gaussian shape. Under each cloud, identify whether scale, covariance or distributional shape is the affected property. Do not imply that a Gaussian is automatically semantically optimal for every task. The figure teaches why one regularizer may not control all properties.

Figure 4.12 - Sliced-Wasserstein comparison.

> **Figure description.** Draw two normalized embedding clouds, choose one unit projection direction, map both clouds onto a line, sort the scalar values, and pair their order statistics with a set of standard-normal target quantiles. Add a second faint projection direction to show that the loss averages over several slices. Label the sorting operation as the exact equal-weight one-dimensional matching step.

Figure 4.13 - Increasing the number of slices.

> **Figure description.** Draw a unit circle or sphere with a small set of sampled directions in one panel and a denser set in the next. Show more coverage of the projection space alongside a larger compute annotation. State in the caption that this is a conceptual illustration of a sampling trade-off, not a convergence guarantee for a particular finite K.

VISReg and SIGReg share the broad goal of distributional regularization, but their statistics are different. SIGReg measures characteristic-function discrepancy along projections. VISReg sorts projected samples and measures a transport discrepancy while handling scale separately. The next section briefly places these ideas alongside related multimodal and probabilistic variants without treating each as a full new architecture family.

4.9. Related JEPA directions: modalities and uncertainty

The image and video examples are the main progression of this chapter. Other JEPA papers extend the same predictive idea to additional modalities or change the mathematical treatment of uncertainty. They are important to recognize, but they do not all define one unified architecture.

4.9.1. Audio prediction

A-JEPA (Fei, Fan and Huang, 2023) applies joint-embedding prediction to audio. Its inputs are represented through time-frequency structure, and its masking strategy accounts for the fact that time and frequency axes have different meanings. Audio-JEPA (Tuncay and colleagues, 2025) is a separate work with a separate method and evaluation. It uses masked audio representations and evaluates learned features on audio tasks. A-JEPA and Audio-JEPA should not be merged because the names are similar.

The general design choice is the same as in image and video JEPA: select a context, define a target representation, and predict the target in the embedding space. But the target geometry changes. A time-frequency patch is not interchangeable with an image patch, and the mask should reflect the structure of the signal.

4.9.2. Vision-language prediction

VL-JEPA (Chen and colleagues, ICLR 2026) predicts continuous embeddings of target text rather than autoregressively generating token sequences during its main predictive training. The model is trained to align visual context with a representation of language. A text decoder is used when a textual answer must be produced. The paper evaluates retrieval, open-vocabulary recognition and visual question answering.

This is a useful demonstration that the target embedding need not be visual. It is not a guarantee that a visual-language embedding contains the physical state needed for manipulation. A language-aligned representation is optimized for cross-modal tasks; control sufficiency remains a separate requirement.

4.9.3. Variational and probabilistic formulations

Two 2026 works use related but distinct names and should be differentiated carefully.

Var-JEPA (Gögl and Yau, ICML 2026) derives a variational formulation by making an underlying coupled latent-variable structure explicit. It introduces an evidence lower bound (ELBO) and studies uncertainty-aware latent prediction. The paper instantiates the approach for tabular data through Var-T-JEPA. Its central claim is that a deterministic JEPA can be understood as a special case of a more explicit variational construction under the paper's model class.

VJEPA (Huang, ICML 2026) is a different paper. It proposes probabilistic prediction distributions over future latent states and connects the model to predictive state representations and Bayesian filtering. The paper also introduces Bayesian JEPA, which combines learned dynamics with modular prior experts through a Product of Experts. It evaluates uncertainty-aware predictions in several benchmark settings, including continuous-control tasks.

The names are easy to confuse. Var-JEPA and VJEPA are separate ICML papers with different authors, methods and empirical scope. Neither should be described as merely a new version of V-JEPA for video. In this chapter, the deterministic JEPA examples are sufficient to develop the main architecture; the variational papers provide a compact pointer toward probabilistic latent prediction.

4.9.4. A compact map of the family

Direction	Target representation	Distinctive emphasis
I-JEPA	Hidden image regions	Spatial context-to-target prediction
V-JEPA	Masked video regions	Spatiotemporal feature prediction
V-JEPA 2	Video features; later robot-conditioned future features	Action-free pretraining followed by action-conditioned post-training
V-JEPA 2.1	Dense image/video token features	Spatially structured and temporally consistent representations
A-JEPA	Audio time-frequency features	Audio-aware context and masking
Audio-JEPA	Audio features	A distinct audio JEPA formulation and evaluation
VL-JEPA	Text embeddings conditioned on vision	Vision-language prediction in embedding space
Var-JEPA	Latent-variable predictive representation	Variational derivation and ELBO
VJEPA	Distribution over future latent states	Probabilistic world models and filtering

Figure 4.14 - JEPA targets across modalities.

> **Figure description.** Place image, video, audio spectrogram and text targets in four panels. In each, show a context input, its encoder, a predictor and a target

> encoder or target embedding. Annotate the structural meaning of each mask: spatial blocks for images, spatiotemporal regions for video, time-frequency patches for audio, and text embeddings for vision-language prediction. Add a visually separate probabilistic branch showing a distribution over latent futures rather than a single point prediction. Do not imply that every method shares the same architecture or training details.

The shared idea is prediction in a representation space. The modality-specific encoder, target construction, masking strategy, regularizer and uncertainty treatment remain method-specific.

4.10. Action-conditioned prediction and planning

Passive video prediction is not yet a model of an agent's interventions. If a robot observes a cup moving across a table, the frames alone may not reveal whether the motion came from the robot, another person, gravity or an external disturbance. To predict the consequence of a candidate action, the model must receive action information and be trained on data that connect actions to subsequent observations.

An action-conditioned world model predicts a future representation from a history of observations and controls. A schematic form is

$$[\hat{z}_{t+1:t+H} = g_\phi(z_{t-L:t}, a_{t:t+H-1}, s_{t-L:t}),$$

where z denotes visual features, a the action sequence, and (s) optional proprioceptive or end-effector state. The exact history length, action representation, conditioning mechanism and rollout target differ across implementations.

This extra input changes the identification problem. Passive pretraining can learn regularities in how the world usually evolves. Action-conditioned training must learn how the predicted future changes when the action changes. A model can be accurate on observed trajectories and still fail to answer counterfactual questions if the training data do not cover the relevant action-state combinations.

4.10.1. From prediction to planning

Once a model can roll candidate actions forward in latent space, a planner can evaluate possible futures without decoding every pixel. Given a current latent state z_t, candidate action sequence $mathbf a_{t:t+H-1}$, and goal representation z_g, a basic objective is

$$[\mathbf{a}^* = \arg\min_{\mathbf{a}} \mathcal{C}\big(\hat{z}_{t+1:t+H}(\mathbf{a}), z_g\big) + \lambda \mathcal{R}(\mathbf{a}).$$

The first term measures predicted compatibility with the goal; the second can penalize action magnitude, infeasible controls or other constraints. This is a conceptual decomposition, not a universal objective used by every JEPA system.

A planner usually optimizes a sequence but executes only the first action or a short prefix. It then receives a new observation, updates its state estimate, and replans. This is model-predictive control. Replanning limits the period during which the system relies on an open-loop prediction, but does not remove errors in the learned model or guarantee safety.

Several design choices interact:

- **Horizon:** a longer rollout gives the planner more foresight but allows prediction errors to compound.
- **Action representation:** low-level motor commands, end-effector deltas and temporally extended actions define different control problems.
- **Proprioception:** robot state may disambiguate visually similar scenes or indicate the current configuration of the arm.
- **Predictor architecture:** temporal context, action conditioning and normalization determine how the model uses its inputs.
- **Planner:** random shooting, the cross-entropy method, gradient optimization and other optimizers have different assumptions and failure modes.
- **Goal and cost:** latent distance is a proxy for task success. It may ignore collisions, force constraints, hidden state or irreversible outcomes.

A strong planning result is therefore a result about a complete system: encoder, predictive model, action data, planner, objective, environment and feedback loop.

4.10.2. V-JEPA 2-AC as a staged example

V-JEPA 2-AC makes the division between representation and dynamics explicit. The V-JEPA 2 visual encoder is first trained with action-free video and image data. The second stage freezes the visual encoder and trains an action-conditioned predictor using robot trajectories. The predictor uses past frame features, robot actions and end-effector states to estimate future frame features.

The post-training data are much smaller than the passive pretraining corpus. The V-JEPA 2 paper reports less than 62 hours of unlabeled robot video from DROID for the action-conditioned stage. The system is deployed on Franka arms in two laboratories and plans toward image goals. Its zero-shot claim is tied to the evaluated deployment environments and tasks; the model still uses robot interaction data during post-training.

This staged strategy separates two forms of knowledge. Large-scale passive video pretraining

builds broad visual representations. Robot trajectories add information about the relation between control variables and future observations. The action-conditioned predictor is not obtained simply by relabelling an action-free predictor as a world model.

4.10.3. The broader JEPA-WM design space

The JEPA-WMs study by Terver and colleagues (2025/2026) investigates which model and planning choices contribute to physical planning performance. It examines visual encoders, predictor architecture and depth, the number of rollout steps used during training, context length, proprioception, model scaling, and planning optimizers. The paper evaluates simulated environments and real-robot data, and combines its ablations into a proposed model that outperforms the DINO-WM and V-JEPA 2-AC baselines on the reported navigation and manipulation evaluations.

This is valuable because it shifts attention from a single pretraining objective to the entire model-planner interface. Multi-step rollout training is intended to improve predictions beyond the immediate next step. Proprioception can help the predictor distinguish states that appear similar visually. The choice of planner determines how effectively the learned model can be searched. The results remain tied to the paper's experimental tasks and baselines.

The official JEPA-WMs repository contains code, datasets and checkpoints. Its license is CC BY-NC-ND 4.0, so code and model artifacts should not be copied into a commercial textbook. The chapter can describe the architecture and reported experiments in its own words, with attribution, without reproducing restricted code.

4.10.4. Energy-based prediction and planning

Energy-based models provide a complementary way to score compatibility. An energy function (E(x,y)) assigns a scalar to a pair or configuration. Lower energy means the configuration is preferred under the model's learned scoring convention. An energy function is not automatically a normalized probability distribution. To obtain a Gibbs density, one would need a partition function:

$$[p(y \mid x) = \frac{\exp[-E(x,\, y)]}{Z(x)}, \qquad Z(x) = \int \exp[-E(x,\, y')]\, dy' .$$

For complex continuous spaces, computing (Z_x) can be intractable. Many energy-based systems therefore use the energy as a score or objective without explicitly evaluating a normalized density.

In a predictive world model, energy can score whether a proposed future representation is compatible with the context or whether a trajectory satisfies a desired constraint. In an agent architecture, a cost can instead express a task objective or discomfort. These uses are related but

not identical. A prediction loss measures mismatch to a training target; an energy scores compatibility; a task cost ranks futures according to goals. The same scalar notation should not hide these different semantics.

EB-JEPA is an official Meta FAIR library and tutorial for JEPA-style representation learning, prediction and planning. Its examples include image JEPA, video JEPA and action-conditioned video JEPA for planning in a Two Rooms environment. The repository is Apache-licensed. It is a useful executable reference, but it should be described as a library and collection of examples, not casually equated with a separate foundational theorem about energy-based learning.

Figure 4.15 - From action-conditioned rollout to receding-horizon planning.

> **Figure description.** Draw the current observation and proprioceptive state entering a frozen visual encoder. Feed the resulting latent history and several candidate action sequences into an action-conditioned predictor. Show each sequence producing a distinct predicted latent trajectory. A goal encoder supplies a target representation; the planner scores the candidate rollouts and selects the best sequence. Only the first action is sent to the robot. A feedback arrow returns the new camera observation and robot state before the next planning cycle.

Figure 4.16 - Prediction loss, energy and task cost are different quantities.

> **Figure description.** Use three side-by-side panels. The first compares a predicted target embedding with the target embedding and labels the distance as a training loss. The second assigns lower and higher compatibility scores to candidate latent configurations and labels them energy. The third compares future trajectories against a task goal or constraint and labels the scalar as task cost. Use separate symbols and units or explicitly say they need not share a scale. The figure should prevent readers from treating all scalar objectives as interchangeable.

Action-conditioned prediction makes latent world models useful for planning, but an agent also needs to decide what counts as a good future, retain relevant history, and configure its modules for the task. The next section presents LeCun's proposed architecture for organizing these capabilities.

4.11. A path towards autonomous machine intelligence

In *A Path Towards Autonomous Machine Intelligence* (2022), Yann LeCun proposes a cognitive

architecture for agents that learn to predict, reason and plan from observation and interaction. The paper is explicit about its status: it is a position paper expressing a research vision, not a conventional scholarly article reporting one complete implemented system. Its value for this chapter is architectural. It places predictive world models inside a wider agent that has perception, memory, intrinsic objectives, an actor and a mechanism for configuring the system.

The proposal should therefore be taught as a design framework. Later systems such as V-JEPA 2-AC and JEPA-WMs instantiate parts of the broader idea, but they should not be presented as complete implementations of every module in the 2022 proposal.

4.11.1. The six-module architecture

The architecture has six top-level modules:

1. **Perception** estimates the current state of the world from observations.
2. **World Model** predicts possible future states under imagined action sequences.
3. **Cost** evaluates current and predicted states through a scalar energy.
4. **Actor** proposes action sequences and ultimately sends actions to effectors.
5. **Short-Term Memory (STM)** stores relevant past, current and predicted states together with associated intrinsic costs.
6. **Configurator** configures the other modules for the task at hand.

The Cost module contains two submodules, not two additional top-level modules. The **Intrinsic Cost** is described as immutable and computes immediate intrinsic energy from a state. The **Trainable Critic** estimates future intrinsic cost. Their outputs contribute to the Cost module's scalar energy.

This hierarchy matters in the figure. Cost is one of the six peers in the architecture. Intrinsic Cost and Critic belong inside it.

4.11.2. Perception: estimate the current state

The Perception module transforms observations into an internal state representation. It need not preserve every pixel or sensor value. It should retain what matters to the configured task and to future predictions. Its state estimate is the interface used by the Actor, World Model and Cost.

This links the architecture to the representation contract from Chapter 2. The state is task-dependent: a representation adequate for object recognition may omit velocity, contact state or another variable needed for manipulation. The Configurator can modulate Perception so that the representation serves the task being performed.

4.11.3. World Model: imagine possible futures

The World Model predicts future world states as a function of imagined action sequences proposed by the Actor. It is not merely a next-frame predictor. The proposed agent needs to represent multiple plausible futures because the world may be partially observed, stochastic, chaotic or influenced by other agents.

The paper emphasizes two open requirements: representing uncertainty and multiple possible predictions, and learning a predictive model that can support planning. A deterministic point predictor may be inadequate when several futures remain plausible. The architecture leaves room for hierarchical JEPA models that predict at multiple levels of abstraction and over different time horizons.

The World Model also interacts with STM. It can retrieve relevant past states, store new states, predict futures, and use memory when completing or correcting an estimate of the current world. This makes memory part of predictive computation rather than an isolated log of the past.

4.11.4. Cost: intrinsic energy and a trainable critic

The Cost module emits a scalar called energy. In the proposal, the objective is to prefer states with lower long-run intrinsic cost. The module has two components:

- **Intrinsic Cost** is immutable and maps a current or predicted state to an immediate intrinsic energy. LeCun's examples include pain, pleasure, hunger, satiety, curiosity and other basic drives.
- **Trainable Critic** predicts future values of intrinsic cost. It can use states retrieved from memory and is trained to estimate the longer-term consequences associated with a state.

The immediate cost and the critic's prediction have different roles. The intrinsic term defines the basic drives or preferences. The critic estimates how those drives are likely to evolve. Their combination provides a scalar signal that can evaluate imagined trajectories.

The term *energy* should not be misread as an automatically normalized probability. In this architecture it is a cost-like scalar used for ranking states and action sequences. A Gibbs probability would require an explicit normalization and a defined base measure. The paper's use of energy is compatible with an optimization objective without claiming that every energy value is a calibrated probability.

A second distinction is between this agent-level Cost and the predictive loss used to train a JEPA encoder. The prediction loss encourages an embedding to match a target. Intrinsic energy evaluates a state according to the agent's drives. A training loss, a compatibility energy and a task cost can all be scalar quantities, but they represent different functions.

4.11.5. Actor: propose actions and optimize trajectories

The Actor proposes action sequences. The World Model predicts the outcomes associated with those sequences and sends the predicted states to Cost. Cost evaluates the expected energy of each candidate. The Actor can use the resulting gradient with respect to the proposed actions to optimize a sequence, at least when the relevant modules are differentiable. For discrete actions, the paper discusses alternatives such as dynamic programming.

The system sends the first action or a short prefix to the effectors, observes the result, updates its state estimate and repeats the process. This is closely related to model-predictive control. It limits commitment to an open-loop sequence while retaining the ability to plan over a longer horizon.

The architecture assumes differentiability along the paths used for gradient-based planning. The external physical environment is not a differentiable module through which gradients can generally be back-propagated. The system can differentiate through its learned Perception, World Model, Cost and Actor components, then execute an action and obtain a new observation from the real environment. The diagram must not suggest that gradients pass through the robot or physical world itself.

4.11.6. Short-Term Memory: associative access to relevant states

STM stores relevant past, current and predicted states together with associated intrinsic costs. The World Model can query memory, retrieve values and store new states during temporal prediction or spatial completion. The Critic can retrieve past states and their observed intrinsic costs when learning to estimate future cost.

The paper describes an associative memory inspired by key-value memory networks. The point is not simply to append a longer observation history to a transformer. An associative memory can retrieve a past state that is relevant to the current query even when it is not adjacent in time. This is especially useful when an agent encounters recurring configurations or must recall a state whose consequences were previously experienced.

The label *Short-Term* should not be taken to mean that memory only stores a fixed number of recent frames. The proposal describes memory over past, current and predicted states. Its exact capacity, retrieval rule and update policy remain open implementation choices.

4.11.7. Configurator: task-dependent executive control

The Configurator is an executive module. Given a task, it configures Perception, World Model, Cost and Actor. It can modulate the parameters of the modules it controls, choosing which state variables matter, which predictions should be made, what costs define success, and which policy or planner should be used.

This is not equivalent to adding a task embedding to a fixed network. The proposed role is broader: configuring the functional behaviour of several modules for the task at hand. The Configurator also receives information from the other modules in the architecture. It is therefore not simply a one-way instruction input.

4.11.8. A proposal, not a completed blueprint

The six-module architecture is valuable because it forces several responsibilities to be separated. A predictive representation is not a goal. A goal is not a controller. A controller is not memory. A memory is not a mechanism for deciding which problem should be solved. By making these modules explicit, the proposal exposes the research needed to connect representation learning with autonomous behaviour.

The framework does not specify a finished algorithm for every module or guarantee that joint differentiable optimization will be stable. It does not settle how intrinsic drives should be chosen, how the Critic should generalize, how memory should be retrieved, how multiple futures should be represented, or how a Configurator should learn to reconfigure a complex agent. Those are research questions rather than solved details.

Figure 4.17 - LeCun's proposed architecture for autonomous intelligence.

> **Figure description.** Draw the six top-level modules with Perception at the left, Actor near the lower centre, World Model at the upper right, Cost as a single module on the lower right, Short-Term Memory near the top centre, and Configurator above. Place Intrinsic Cost and Trainable Critic inside the Cost boundary. Draw the environment outside the agent: Actor sends an action to it and it returns a percept to Perception. Show Perception feeding Actor, World Model and Cost; Actor feeding World Model and Cost; World Model feeding Cost; and bidirectional information flow between World Model and STM. Show Configurator modulating Perception, STM and World Model, and note that its task configuration affects the other modules. Use internal arrows for differentiable computation and make clear that no gradient passes through the physical environment. The figure must reproduce the logical topology, not copy the source artwork.

The architecture gives a place to the representation-learning ideas in this chapter. JEPA provides a family of predictive mechanisms for the World Model. Action conditioning connects prediction to interventions. Cost and Actor use imagined futures to choose actions. STM supplies relevant experience, while Configurator selects the task-dependent behaviour.

4.12. Mode 1, Mode 2 and hierarchical planning

LeCun's proposal distinguishes two ways an agent can select an action. Mode 1 produces an action reactively from the current state, possibly using short-term memory. Mode 2 uses the World Model and Cost to imagine candidate action sequences and search for a lower-cost future. These modes are not two different kinds of intelligence in a strict psychological taxonomy. They are two computational operating modes in the proposed architecture.

4.12.1. Mode 1: reactive behaviour

In Mode 1, Perception estimates the current state s_t, and a policy component of the Actor maps that state to an action:

$$[a_t = A(s_t).$$

The policy may also read STM, for example to retrieve an action associated with a similar past state. It need not run a multi-step planning procedure for every action. The Configurator adjusts the policy for the task.

Mode 1 is computationally inexpensive at inference time because it can use a single forward pass through a policy. It can be sophisticated and can incorporate memory. Its defining feature is that it does not deliberately search through imagined action sequences using the World Model and Cost on each decision.

The proposed architecture can still update the World Model while operating in Mode 1. It can predict the next state, execute the selected action, receive the next percept and use the observed state as a target for predictive learning. Thus, reactive action selection does not imply that the model ceases to learn from transitions.

4.12.2. Mode 2: reason by simulating and evaluating

Mode 2 uses the predictive model as an internal simulator. A typical episode follows this sequence:

1. Perception estimates the current state and Cost evaluates its immediate energy.
2. Actor proposes an initial action sequence.
3. World Model predicts one or more possible state sequences under those actions.
4. Cost evaluates the predicted states, often by accumulating per-step energy.
5. Actor changes the proposed actions to reduce the estimated total cost.
6. After optimization or search, the system executes the first action or a short prefix.
7. The next observation updates the state, and the process repeats.

A deterministic shorthand is

$$[s_{t+1} = F_\theta(s_t, a_t), \qquad J(\mathbf{a}) = \sum_{t=1}^{H} C(\hat{s}_t).$$

The Actor searches for an action sequence with low predicted cost. If the learned model and cost are differentiable, gradients can be propagated through the imagined rollout to the action variables. If actions are discrete or the objective is discontinuous, dynamic programming, combinatorial search, tree search or gradient-free optimization may be more appropriate.

The real environment is not part of this differentiable computation graph. The agent differentiates through its learned model and cost, then executes an action in the world and obtains a new observation. This distinction is central to why a world model is useful: it permits many candidate sequences to be evaluated internally without physically executing all of them.

Mode 2 resembles model-predictive control. A plan is optimized over a horizon, but only its first action or a short prefix is executed. New observations are used to re-estimate the state and replan. This feedback loop limits the duration of open-loop commitment, although it cannot prevent systematic model error or guarantee safe execution.

4.12.3. Distilling deliberation into a reactive skill

Repeatedly running Mode 2 can be expensive. The World Model is a shared reasoning resource, and deliberative search repeatedly rolls it forward for candidate action sequences. LeCun proposes using the outputs of Mode 2 to train a reactive policy.

Let $\tilde{a}_t$ be the action selected by Mode-2 optimization for state s_t. A policy A_ψ can be trained to imitate it:

$$[\mathcal{L}_{policy} = \mathbb{E}_{s_t} D\big(\tilde{a}_t, A_\psi(s_t)\big),$$

where D is an action-space divergence or regression loss appropriate to the action representation. Once trained, the policy approximates the expensive planning result with a direct state-to-action mapping. This is a form of amortized inference: compute a costly answer for sampled states, then learn a faster function that approximates those answers.

The reactive policy can be used directly in Mode 1, or to initialize an action sequence before Mode 2 refines it. The proposal therefore does not require choosing one mode permanently. Deliberative planning can create or improve a skill, while a reactive policy executes that skill efficiently.

This is related to policy distillation and imitation learning, but the teacher here is the agent's own planner. The target action is produced by minimizing the model's estimated cost, not necessarily by copying an external expert. It inherits the planner's model bias and cost design: a policy can only imitate the plan that the learned system considers good.

4.12.4. Hierarchical JEPA and multiple time scales

The proposal extends JEPA from a single predictive mapping to a hierarchy of representations and predictive models. A lower level can represent detailed perceptual states and short motor actions. Higher levels can represent temporally extended events, abstract state changes or action segments. A high-level model can predict a more distant target in a coarser representation, while a lower-level model predicts the intermediate states needed to realize that target.

One way to express the idea is to let $s_t^{(0)}$ denote a fine-scale state and $s_t^{(1)}, s_t^{(2)}, \ldots$ denote increasingly abstract states. Each level predicts a target at its own time scale:

$$\hat{s}_{t+\Delta_k}^{(k)} = F_k(s_{\leq t}^{(k)}, a_{t:t+\Delta_k}).$$

This is schematic notation for a hierarchy, not a canonical implementation specification. The important structure is that a high-level plan can constrain the subgoals or action sequences of lower levels, while the lower levels provide the detailed predictions required to carry out the plan.

Hierarchical prediction is motivated by long-horizon planning. A planner that reasons directly over every low-level control step may face an enormous search space. Temporally extended representations and action chunks can shorten the effective horizon at the high level. Yet abstraction creates its own risk: if a high-level state discards a variable needed by a low-level controller, the plan may be impossible to execute.

4.12.5. Multiple futures and uncertainty

A deterministic model returns one predicted trajectory for a given starting state and action sequence. But the future can be uncertain for several reasons:

- **Aleatoric uncertainty:** the environment itself is stochastic.
- **Partial observability:** the state representation does not contain all relevant information.
- **Epistemic uncertainty:** the learned model is imperfect because of limited data, model capacity or optimization.
- **Other agents:** other people or systems may make decisions that change the outcome.

The proposal argues that a World Model should represent multiple plausible futures rather than collapse them into a single prediction. This is particularly important when the action sequence does not uniquely determine the outcome. A model that averages mutually exclusive futures can produce a latent state that corresponds to neither outcome.

There are several possible ways to address this problem, including latent-variable models, ensembles, stochastic predictors or explicit distributions over future embeddings. These approaches have different assumptions and computational trade-offs. The 2022 proposal does not

prescribe one definitive solution. The later VJEPA and Var-JEPA papers provide probabilistic directions, but they do not solve every uncertainty problem in physical planning.

Figure 4.18 - Mode 1, Mode 2 and the distillation loop.

> **Figure description.** Draw Mode 1 as a direct path from Perception state through a reactive policy to an action. Draw Mode 2 beneath it as a loop: Actor proposes an action sequence, World Model rolls out candidate states, Cost scores them, and an optimizer updates the actions. Show the optimized action as a training target for the reactive policy. Include a second arrow indicating that the policy can initialize later Mode-2 search. Label the policy as amortizing part of the planning computation, not replacing the world model.

Figure 4.19 - Hierarchical predictions at multiple time scales.

> **Figure description.** Draw a high-level latent state predicting an abstract future subgoal over a long interval. Under it, show a lower-level predictor expanding that subgoal into intermediate states and motor actions. A still finer control layer should execute the short action sequence and report the resulting state upward. Use different horizontal time scales and clearly label the hierarchy as a proposed organization, not a standardized JEPA architecture.

Figure 4.20 - A single future versus a distribution of futures.

> **Figure description.** From the same current latent state and candidate action, branch to several possible future trajectories. Show one deterministic predictor collapsing these alternatives into one point estimate, then show a probabilistic or multi-hypothesis predictor retaining separate modes. Do not imply that uncertainty is automatically calibrated. Label aleatoric, partial-observability and model uncertainty as distinct possible sources.

The distinction between Mode 1 and Mode 2 closes the loop from representation learning to behaviour. A model can learn predictive features, imagine candidate futures, evaluate them and use those evaluations to improve a policy. The proposal’s deeper research challenge is to make this process reliable across long horizons, multiple levels of abstraction and uncertainty.

4.13. Reading a miniature JEPA system

This section turns the chapter's architecture into an implementation workflow. It does not introduce a new experiment or claim a reproduction. The aim is to give the reader a stable mental model for reading official implementations and for identifying which parts of a JEPA are method-defining rather than incidental engineering.

The official I-JEPA repository is a useful reference for image masking, target encoders, predictor queries and distributed training. Its source tree separates the training loop, model definitions, mask collators, data transforms and experiment configurations. The official V-JEPA 2 repository contains separate training and evaluation paths for action-free pretraining, V-JEPA 2-AC post-training and V-JEPA 2.1. The separation in the repository mirrors a substantive difference in the methods.

4.13.1. A generic teacher-student JEPA training step

For an EMA-target JEPA such as the image and video examples discussed earlier, one training step can be summarized as follows:

1. Sample an observation or clip.
2. Sample a context and one or more target regions.
3. Encode visible context tokens using the trainable context encoder.
4. Encode target tokens using the slowly updated target encoder.
5. Build a query containing the target's position or time information.
6. Predict target embeddings from context embeddings and the query.
7. Compute a feature-space prediction loss.
8. Add any method-specific regularization and update the trainable parameters.
9. Update the target encoder with the method's EMA rule.

The following pseudocode is deliberately generic. It is not copied from the I-JEPA or V-JEPA repositories and should not be mistaken for a runnable implementation.

```
context, target, query = sample_context_and_target(batch)

z_context = context_encoder(context)
with no_gradient():
    z_target = target_encoder(target)

z_prediction = predictor(z_context, query)
loss = feature_distance(z_prediction, z_target)
loss = loss + regularizer_if_used(z_context, z_target)

update(context_encoder, predictor, loss)
target_encoder = ema_update(target_encoder, context_encoder)
```

The pseudocode assumes a target branch that is updated by EMA. It does not describe every JEPA. LeJEPA uses multi-view prediction and SIGReg without the same EMA teacher-predictor arrangement. VISReg changes the regularization term. V-JEPA 2-AC uses an action-conditioned temporal predictor and a separate post-training stage. A diagram or algorithm that silently combines these recipes would describe a method that none of the cited papers actually proposes.

4.13.2. The data pipeline carries the inductive bias

A training loop can look generic while the mask sampler determines most of the task. For I-JEPA, the relation between spatially distributed context and larger target blocks is central. For V-JEPA, targets and visible context are spatiotemporal. For V-JEPA 2-AC, the action and robot-state sequence must be aligned with the visual feature history and the future targets. The sampler defines what the predictor is expected to infer.

The reader should therefore inspect the data pipeline before treating the loss as the whole algorithm. Check the clip sampling rate, temporal stride, image resolution, crop policy, target length, number of masks, overlap between context and target, and action/observation synchronization. These choices alter the prediction problem.

4.13.3. What must be inspected in an official implementation

When reading a repository, follow one training batch all the way through:

- **Input:** how videos or images are decoded, sampled and normalized.
- **Masking:** how context and target regions are generated and encoded.
- **Encoder:** whether weights are shared, copied or EMA-updated.
- **Predictor:** which position, time or action variables condition it.
- **Target path:** where gradients are stopped and where they are allowed.
- **Loss:** whether it is pointwise feature regression, a multi-view objective, SIGReg, VISReg or another criterion.
- **Optimizer:** which parameters are updated by the gradient step.
- **Target update:** whether an EMA update is present and its schedule.
- **Evaluation:** whether features are frozen, linearly probed, fine-tuned or used in a planning system.

These checks prevent several common reading errors. An EMA target is not the same as a frozen pretrained backbone. A target mask is not necessarily a future-time mask. A predictor is not necessarily a pixel decoder. A training loss is not a planning cost. And a reported frozen-backbone result is not a test of closed-loop control.

4.13.4. A method comparison through its training graph

The following table gives a concise map of the implementation choices developed in this chapter.

Method	Context and target	Predictor / target update	Additional constraint
I-JEPA	Spatial context and multiple image target blocks	Position-aware predictor; EMA target encoder	Mask design is central
V-JEPA	Visible and masked spatiotemporal video tokens	Video feature predictor; target branch follows the paper's teacher design	Video mask and temporal sampling
V-JEPA 2	Video and image pretraining with masked latent prediction	Action-free encoder and predictor	Large-scale passive video/image data
V-JEPA 2-AC	Past visual features, actions and robot state predict future features	New action-conditioned autoregressive predictor on frozen visual encoder	Robot trajectories; rollout training
V-JEPA 2.1	Dense supervision on visible and masked tokens	Deep supervision across encoder depth	Multimodal tokenizers and dense target loss
LeJEPA	Multiple global and local views	View-centre prediction; no EMA teacher/predictor in the specified recipe	SIGReg
VISReg	View-based predictive embedding objective	Method-specific prediction term	Scale, shape and centering regularization

4.13.5. A small implementation exercise

A reader can understand the architectural contract without reproducing the large-scale

experiments. Consider a small image or synthetic video dataset. Specify a context sampler, a target sampler and a query representation. Decide whether the target is updated by EMA or by a different mechanism. Write down the loss and state exactly which branch receives gradients. Then define one downstream evaluation that tests whether the predicted features retain the distinction the task requires.

The exercise is about writing down the model completely, not about claiming that a small run validates the reported results of I-JEPA or V-JEPA. A meaningful reproduction requires the original data, model scale, optimization, training duration and evaluation protocol. This chapter does not run new experiments or present a new empirical result.

4.13.6. License and code-reuse boundaries

The official I-JEPA and V-JEPA 2 repositories are the most relevant references for the implementations described here. EB-JEPA is an Apache-licensed library and tutorial with image, video and action-conditioned planning examples. The JEPA-WMs repository states a CC BY-NC-ND 4.0 license, and the VISReg project's code and weights state CC BY-NC 4.0. The latter two should not be copied into a commercial manuscript without resolving the license restrictions and obtaining permission where required.

The chapter's pseudocode is written independently as explanatory notation. It does not reproduce code from those repositories. For a production reproduction, the reader should follow the cited official repository and its current configuration rather than treating the compact pseudocode as a replacement.

Figure 4.21 - Reading the implementation as a dataflow graph.

> **Figure description.** Draw a left-to-right training pipeline containing decoded observation, sampler, context encoder, target encoder, predictor, feature loss and optimizer. Put the target EMA update below the optimizer as a separate state update. Place the mask sampler above the input paths and connect its outputs to the context and target branches. Use dashed boxes for method-specific regularizers, and annotate that LeJEPA and VISReg substitute different training and regularization structures rather than simply adding another loss to an identical EMA model.

This implementation view reinforces the central principle of the chapter: the name JEPA denotes a design family. To understand an individual method, inspect the full contract between its data, target, predictor, representation geometry, optimization and evaluation.

4.14. Summary and exercises

4.14.1. Summary

A Joint-Embedding Predictive Architecture learns by predicting target representations from context representations. Unlike pixel reconstruction, this objective evaluates a prediction in feature space. The method's meaning depends on how context, target, query, encoder, predictor and regularizer are defined.

I-JEPA applies the idea to masked image regions. Its target blocks and spatially distributed context determine what can be inferred. V-JEPA extends masked feature prediction to video and learns representations useful for appearance- and motion-centric tasks. Neither action-free I-JEPA nor V-JEPA should automatically be described as a model of interventions.

V-JEPA 2 separates action-free pretraining from action-conditioned post-training. V-JEPA 2-AC receives robot actions and end-effector state and predicts future visual features, enabling a planner to search toward image goals under the reported setup. V-JEPA 2.1 is a separate dense-feature learning recipe with visible-token and masked-token loss, deep supervision, multimodal tokenizers and scaling.

Distributional regularization addresses more than trivial collapse. VICReg controls variance and pairwise covariance. LeJEPA motivates an isotropic Gaussian target through a specific downstream prediction-risk analysis and uses SIGReg to compare projected empirical characteristic functions with a standard-normal target. The finite, resampled projections are a practical sketch, not a population-level certificate. VISReg decouples scale and shape: a variance term controls the coordinate-wise spread, a sliced-Wasserstein term compares sorted projections with Gaussian quantiles, and a centering term controls the batch mean. Its scalar scale gradient should not be mistaken for a universal lower bound on every network parameter gradient.

Other JEPA directions extend the target to audio, language or probabilistic latent states. A-JEPA and Audio-JEPA are separate works. VL-JEPA predicts text embeddings from visual context. Var-JEPA and VJEPA are separate variational papers, not versions of V-JEPA.

Action-conditioned prediction links a latent world model to planning. A planner searches over actions using predicted future representations and a goal or cost. Energy, prediction loss and task cost are different quantities. A scalar energy is not automatically a normalized probability.

LeCun's six-module architecture places Perception, World Model, Cost, Actor, Short-Term Memory and Configurator inside one proposed agent design. Intrinsic Cost and the Trainable Critic are submodules within Cost. Mode 1 uses a reactive policy. Mode 2 performs deliberative planning through the World Model and Cost. Mode-2 plans can train a faster Mode-1 policy by imitation. Hierarchical JEPA and uncertainty-aware futures are proposed routes to long-horizon planning, not solved guarantees.

The chapter's main conclusion is that predictive representations are a component of a world model, and a world model is a component of an autonomous agent. Reliable action requires additional machinery: action-linked data, predictive dynamics, goals or costs, planning, feedback, memory and task configuration.

4.14.2. Exercises

1. **Spatial versus temporal prediction.** Give an example in which predicting a hidden image region is useful but does not require predicting the future. Give a second example in which a future frame cannot be predicted adequately without temporal context. Explain how the target sampler changes the task.
2. **Conditional mean.** Starting from the squared-error risk, derive why the conditional mean minimizes expected prediction error. Construct a two-mode target distribution for which the conditional mean is not one of the plausible outcomes. Explain why a deterministic latent predictor can inherit this limitation.
3. **I-JEPA masks.** Consider two context masks with equal area: one contiguous crop and one spatially distributed selection. Give a scene configuration for which the distributed context may provide more useful evidence for a distant target block. State a situation in which the contiguous crop might instead be preferable.
4. **Covariance is not shape.** Construct two two-dimensional distributions with the same zero mean and identity covariance but different higher-order structure. Explain what a variance/covariance regularizer can and cannot distinguish.
5. **SIGReg.** Starting from the empirical characteristic function, write the Epps-Pulley statistic for a batch of scalar samples. Explain why the characteristic function is differentiable with respect to the samples. Why does a finite set of projected directions not prove exact equality of two multivariate distributions?
6. **SIGReg versus VISReg.** Compare the statistics used by SIGReg and VISReg. Identify the role of random directions in each method. Explain why VISReg detaches the standard deviation in the shape normalization and why this does not by itself prove a universal lower bound on encoder-parameter gradients.
7. **Action-conditioned planning.** Write a schematic latent rollout objective for a goal-conditioned robot task. Identify the roles of the encoder, action-conditioned predictor, goal representation, action penalty and planner. Explain why a planner that minimizes only latent distance may still propose an unsafe action.
8. **Mode 1 and Mode 2.** Describe a task that could be learned first through Mode-2 planning and then executed by a Mode-1 policy. Write a distillation loss from planner-generated action targets to a reactive policy. Discuss how model bias in the Mode-2 planner can affect the

distilled skill.

9. **Memory and configuration.** Give one example where associative STM retrieval could provide information missing from the current observation. Give a second example where changing the task should alter the configured perception or cost function.
10. **Uncertainty.** Distinguish aleatoric, epistemic and partial-observability uncertainty in a physical world model. Explain why averaging two possible futures can produce a prediction that is not physically realizable.

4.15. References

Primary JEPA papers

1. Assran, M., Duval, Q., Misra, I., Bojanowski, P., Vincent, P., Rabbat, M., LeCun, Y., and Ballas, N. (2023). ``Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture." Proceedings of CVPR 2023. arXiv:2301.08243.
2. Bardes, A. et al. (2024). ``Revisiting Feature Prediction for Learning Visual Representations from Video." arXiv:2404.08471.
3. Assran, M. et al. (2025). ``V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning." arXiv:2506.09985. The source PDF is mirrored in agent_memory_mustread/core_papers/assran-etal-2025-vjepa2.pdf.
4. Mur-Labadia, L. et al. (2026). ``V-JEPA 2.1: Unlocking Dense Features in Video Self-Supervised Learning." arXiv:2603.14482.
5. LeCun, Y. (2022). ``A Path Towards Autonomous Machine Intelligence." Version 0.9.2, 27 June 2022. OpenReview: BZ5a1r-kVsf.

Regularization and representation geometry

6. Bardes, A., Ponce, J. and LeCun, Y. (2022). ``VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning." ICLR 2022. arXiv:2105.04906.
7. Balestriero, R. and LeCun, Y. (2025). ``LeJEPA: Provable and Scalable Self-Supervised Learning Without the Heuristics." arXiv:2511.08544.
8. Wu, H., Balestriero, R. and Levine, M. (2026). ``VISReg: Variance-Invariance-Sketching Regularization for JEPA Training." arXiv:2606.02572.

Other modalities and probabilistic formulations

9. Fei, Z., Fan, M. and Huang, J. (2023). ``A-JEPA: Joint-Embedding Predictive Architecture Can Listen." arXiv:2311.15830.

10. Tuncay, L. et al. (2025). ``Audio-JEPA: Joint-Embedding Predictive Architecture for Audio Representation Learning." arXiv:2507.02915.

11. Chen, D., Shukor, M., Moutakanni, T., Chung, W., Yu, L., Kasarla, T., Bolourchi, A., LeCun, Y. and Fung, P. (2026). ``VL-JEPA: Joint Embedding Predictive Architecture for Vision-Language." ICLR 2026. OpenReview.

12. Gögl, M. and Yau, C. (2026). ``Var-JEPA: A Variational Formulation of the Joint-Embedding Predictive Architecture – Bridging Predictive and Generative Self-Supervised Learning." Proceedings of the 43rd ICML, PMLR 306:35490–35513.

13. Huang, Y. (2026). ``VJEPA: Variational Joint Embedding Predictive Architectures as Probabilistic World Models." Proceedings of the 43rd ICML, PMLR 306:47703–47731.

Action-conditioned world models and implementation references

14. Terver, B., Yang, T.-Y., Ponce, J., Bardes, A. and LeCun, Y. (2025; revised 2026). ``What Drives Success in Physical Planning with Joint-Embedding Predictive World Models?" arXiv:2512.24497.

15. Terver, B., Balestriero, R., Dervishi, M., Fan, D., Garrido, Q., Nagarajan, T., Sinha, K., Zhang, W., Rabbat, M., LeCun, Y. and Bar, A. (2026). ``A Lightweight Library for Energy-Based Joint-Embedding Predictive Architectures." arXiv:2602.03604. Official repository: facebookresearch/eb_jepa.

16. Facebook Research. ``I-JEPA." Official PyTorch repository: facebookresearch/ijepa.

17. Meta FAIR. ``V-JEPA 2." Official PyTorch repository: facebookresearch/vjepa2.

18. Meta FAIR. ``JEPA-WMs." Official research repository: facebookresearch/jepa-wms. Its repository license is CC BY-NC 4.0; code and weights are not copied into this manuscript.

Earlier papers used as Chapter 3 context

19. Chen, T., Kornblith, S., Norouzi, M. and Hinton, G. (2020). ``A Simple Framework for Contrastive Learning of Visual Representations." Proceedings of ICML 2020, PMLR 119. Source PDF mirrored in agent_memory_mustread/core_papers/chen-etal-2020-simclr.pdf.

20. Zbontar, J., Jing, L., Misra, I., LeCun, Y. and Deny, S. (2021). ``Barlow Twins: Self-Supervised Learning via Redundancy Reduction." Proceedings of ICML 2021, PMLR 139. Source PDF mirrored in agent_memory_mustread/core_papers/zbontar-etal-2021-barlow-twins.pdf.

Source-file inventory

The original PDFs present in the repository’s core-paper folder at the start of this work were V-JEPA 2, SimCLR and Barlow Twins. The latter two are cited only to establish continuity with

Chapter 3: SimCLR’s augmentation-defined invariance and Barlow Twins’ cross-correlation objective are not retaught here. The V-JEPA 2 PDF is the only JEPA-specific original PDF in that folder. I-JEPA, original V-JEPA, LeJEPA, VISReg and the related papers are cited from their primary arXiv, proceedings or OpenReview sources.

The author’s JEPA notes file is a StackEdit HTML export stored with a Markdown extension. It was consulted as the personal research notebook, but exploratory statements were cross-checked against primary publications. In particular, the chapter narrows Gaussian-optimality and gradient claims to their stated assumptions and evidence.